

OUIAF / ORMB

Complete Interview Notes

Oracle Utilities Application Framework • Oracle Revenue Management & Billing

Contents

1. OUIAF
2. ORMB
3. Record Ownership
4. Table, Fields and Metadata
5. Lookup
6. Feature Configuration
7. Extendable Lookup
8. Characteristic Types
9. MO (Maintenance Object)
10. BO (Business Object)
11. BO — CLOB Field (XML Structure Field)
12. Zones
13. Portals
14. Navigation Option

15. Application Service

16. Menu

17. Business Service

18. Scripts

19. UI Maps

1. OUAF

Main Purpose: OUAF (Oracle Utilities Application Framework) is the base framework on which all Oracle Utilities products are built. We use it to do development and customization using configuration tools instead of writing heavy code from scratch. It gives standard building blocks like Maintenance Objects, Business Objects, Algorithms, Scripts, Zones, and Portals. We mainly work in the CM layer (Customer Modification) so our changes stay safe during upgrades. In simple terms, OUAF is the foundation — we install the framework first, then build or customize the product on top of it.

2. ORMB

Main Purpose: ORMB is a complete product built on top of OUAF, specifically designed for Banking/Finance and Insurance/Healthcare domains to handle revenue management, billing, payments, collections, and complex financial transactions. While products like CC&B and C2M are used for Electricity, Water, and Gas utilities, ORMB is meant for financial services and insurance where billing and revenue processes are more complex. We first install the OUAF framework, then install ORMB on top of it based on the domain requirement.

3. Record Ownership

Main Purpose: Record Ownership tells us who owns a particular component or configuration in the system. It helps us identify what is delivered by Oracle and what is customized by us.

Three Types:

- FW → Owned by OUAF Framework (core framework components)
- Base → Owned by the Product (like ORMB, CC&B, etc.)
- CM → Customer Modification (our custom development). We should always work in the CM layer.

This is very important during upgrades because only CM components are safe and will not be overwritten.

4. Table, Fields and Metadata

Metadata

Main Purpose: In OUAF, Metadata means storing the structure and behavior of the application in database tables instead of hardcoding everything in Java. This is the core strength of the framework. Because of metadata, we can control screens, fields, validations, and business rules mostly through configuration rather than writing heavy code. This makes development faster and upgrades safer. In simple words: most of the application design is driven by data stored in tables (metadata), not by static code.

Table and Fields

Main Purpose: In OUAF, every Maintenance Object (MO) is built using one or more database tables. The Primary Table holds the main record, and Child Tables hold additional related data.

When we create or extend an MO, we define:

- Which tables are part of it
- The relationship between tables (using Parent Constraint)
- Which fields belong to which table

This structure allows the framework to automatically handle insert, update, delete, and validation logic for that object.

Key Point: We should not directly modify base tables. For custom fields, we usually extend the MO or use characteristics / extendable lookups.

5. Lookup

Main Purpose: Lookup is used to define a fixed list of valid values for a field. Instead of allowing free text, we restrict the user to select only from the values we define in the lookup. It is mainly used for simple dropdown values where we don't need extra properties.

Example: City Lookup containing values like HYDB, BNGL, CHNI. We can directly use this lookup wherever we need a City field with limited options.

6. Feature Configuration

Main Purpose: Feature Configuration is built on top of Lookup. It allows us to add extra configuration values against each lookup value. It gives more flexibility than a normal lookup when we need to store additional details for each option.

Example: If we have a City Lookup, using Feature Configuration we can define multiple options for each city (like Hitech City, Gachibowli, Madhapur under Hyderabad) along with sequence numbers. Very useful for dynamic configurations.

7. Extendable Lookup

Main Purpose: Extendable Lookup is used when we need to store additional properties/attributes against each lookup value. It is more powerful than normal Lookup or Feature Configuration. It uses a Business Object (MO: Extendable Lookup) so we can define custom fields and capture more detailed information for each value.

When to use: Use this when a simple lookup or feature configuration is not enough and we need to maintain extra data (like description, effective dates, status, etc.) for every lookup value. This is the most flexible option among the three.

8. Characteristic Types

Main Purpose: Characteristic Types (Char Types) are used to capture additional information or properties for any record (like Person, Account, Bill, etc.) without changing the base database tables. It is a very common and flexible way to store extra data in OUAF/ORMB. We define what kind of extra information we want to capture and how it should be validated.

Four Types:

■ **Predefined Value** — Fixed list of values; user can only select from it (Example: Customer Type — Residential / Commercial / Industrial).

■ **Ad hoc Value** — User can enter any free text value; a validation rule can be added if needed (Example: Remarks or Comments field).

■ **File Location Value** — Used to store a file path or web link (URI) (Example: storing a document/image path related to the record).

■ **Foreign Key Reference** — Value must come from another table in the system (Example: linking to a City table so only valid cities can be selected).

One of the most frequently used configuration options in any OUAF-based product.

9. MO (Maintenance Object)

Main Purpose: A Maintenance Object (MO) is a group of related database tables that together represent one business entity (like Person, Account, Bill, etc.). It defines how data for that entity is stored and maintained.

Key Points:

- Every MO has 1 Primary Table and 0 or more Child Tables.
- The Primary Table holds the main record.
- Child Tables store additional related information (names, addresses, characteristics, etc.).
- Most MOs are delivered by the base product; custom MOs can be created when needed.
- Don't modify fields if the MO is owned by Base or FW.

MO Options

Main Purpose: MO Options control the behavior of Business Objects that use this Maintenance Object.

Key Points:

- BO Maintenance (Y/N): Y = BO can do full operations (Add, Change, Delete, Read). N = BO can only Read (default).
- BO Restrict Field Name: Protects specific fields from being changed through the Business Object.
- These options control what operations are allowed on the MO through Business Objects.

MO Algorithms

Main Purpose: MO Algorithms allow us to plug in business logic at specific system events on the Maintenance Object level — they run regardless of which BO is used, and even if the MO is accessed directly.

Key Points:

- Common Algorithm Spots: Information, Audit, Revision Control, Transition, Transition Error. (In real product builds you'll also commonly see **Determine BO**, used so the framework knows which Business Object should control a given record.)
- Information spot is single event (only one algorithm). Most other spots are multi-event (multiple algorithms run in sequence).
- These run at the MO level and are useful for audit logging, status transition handling, and generating display info strings.

10. BO (Business Object)

Main Purpose: A Business Object (BO) is a logical view of a Maintenance Object. While MO defines the physical tables, BO defines business rules, validations, lifecycle, and how users/systems interact with that entity.

Key Points:

- BO is built on top of one MO.

- Used to add validations, defaults, and business logic without touching the base MO.
- Most custom development and business rules live at the BO level.
- BO supports inheritance (Child BO can inherit from Parent BO).

BO Options

Main Purpose: Control certain behaviors of the Business Object.

Key Points:

- Inactivate Algorithm: stops a particular (inherited/base) algorithm from executing on that BO.
- Not as extensive as MO Options — used for specific overrides rather than broad behavior control.
- Most real control happens through BO Algorithms and Schema.

BO Schema

Main Purpose: Defines the structure of the Business Object in XML format — which fields, groups, and lists are exposed and how they map to the underlying MO tables.

Key Points:

- mapField — maps a single field.
- type="group" with mapXML — groups fields together.
- type="list" with mapChild — maps to child tables.
- Schema is mandatory; it controls what data can be sent/received via the BO.
- Acts as a contract between the BO and external systems/UI.

BO Algorithms

Main Purpose: Attach business logic at different stages while a record is processed through the Business Object.

Key Points:

- Common Algorithm Spots: Information, Audit, Pre-Processing, Validation, Post-Processing.
- Pre-Processing: runs before MO processing, only when BO is invoked.
- Validation: extra validations after MO-level validation.
- Post-Processing: runs after successful processing (e.g., creating a To Do, logging).
- Information: builds the display info string.

BO Life Cycle

Main Purpose: Used when an entity moves through different statuses during its lifetime. Defines possible states and the rules for moving between them.

Key Points:

- Three kinds of states: Initial (exactly one — starting state), Interim (zero or more — middle states), Final (zero or more — ending states).

- Only defined for status-driven BOs.
- Enter, Exit, and Monitor algorithms can be attached on each state:
 - Enter — runs when BO enters that state.
 - Exit — runs when BO leaves that state.
 - Monitor — used by batch to auto-move the BO to the next state.
- Very useful for workflows like approval processes and status transitions.

11. BO — CLOB Field (XML Structure Field)

Main Purpose: Many Maintenance Objects don't store every BO element in its own column. Instead, the MO has one "XML Structure Field" — usually of data type **CLOB** (sometimes **XML**) — where several BO elements are combined and stored together as a single XML document inside that one column. This is the field people commonly refer to as the "BO CLOB field."

Key Points:

- In the BO Schema, we map elements into this field using the mapXML attribute instead of a plain mapField.

Example: `<filePath mapField="F1_FILE_PATH" mapXML="MST_CONFIG_DATA" required="true"/>`

- When the BO record is saved/updated, the framework automatically assembles all elements mapped to that structure into one XML document and stores it in the single CLOB column.
- Limitation: if the MO's XML structure field is of data type CLOB, you typically cannot index it or join on its contents via SQL — so this data isn't easily searchable at the database level.
- If the field is instead defined as data type **XML** (not CLOB), the database does allow querying/searching based on that data, provided appropriate indexes are defined.
- This pattern is commonly seen in MOs that need flexible/variable attributes (e.g., Characteristics, Extendable Lookups, many config-heavy MOs) without adding a new physical column every time a new attribute is needed.

■ **Interview Tip:** If asked "why would an MO store data in a CLOB instead of separate columns?" — answer: it gives flexibility to add new attributes without schema/table changes, but the trade-off is you lose the ability to do SQL-level filtering/joins on that data (unless the column is typed as XML instead of CLOB).

12. Zones

Main Purpose: Zones are the building blocks used to display information on a Portal. A Portal is made up of one or more Zones, used to show data in formats like lists, info, queries, maps, etc.

Key Points:

- Zones are the main way to show data to the user in OUAF/ORMB.
- Two main types: Explorer Zones and Object Map Zones.
- Can be configured with filters, SQL queries, and broadcasting.

1. Explorer Zones (Most Commonly Used) — display data using SQL queries

- Info Data Explorer — Single SQL: one SQL query, used for displaying details on a page.
- Info Data Explorer — Multiple SQL: multiple SQLs for different sections of information.
- Query Data Explorer — Multiple SQL: used for search/list pages where users query and see results.
- Multi Query Zones: multiple zones appear as a dropdown; user switches between queries/zones.

Key Points for Explorer Zones:

- User Filters (F1–F25) and Hidden Filters (H1–H10).

- SQL written in the zone; columns mapped using type=sqlcol.
- Broadcast passes selected values (PER_ID, ACCT_ID) to other zones on the same portal.
- Supports Multi Select (checkboxes) when needed.

2. Object Map Zones — display data using UI Maps (custom HTML screens)

- Explicit Object Map: we directly specify which UI Map to use.
- Derived Object Map: UI Map is automatically derived from the Business Object options.

When to Use: Explorer Zones for SQL-driven data; Object Map Zones for custom-designed screens (HTML/JS/CSS).

User Filters (F1–F25)

Search criteria shown to the user; values feed the SQL via bind variables (:F1, :F2, etc.). Up to 25. Can set labels, mandatory flags, default values. Common for Account ID, Person ID, Date Range, Status, etc.

Hidden Filters (H1–H10)

Pass values to the SQL without showing them on the UI. Up to 10. Useful for passing values from a Broadcast, Portal context, defaults, or another zone on the same page (e.g., passing PER_ID from a Person zone to an Account zone).

SQL Statement in Zones

The actual query that fetches data for the zone. Normal SQL with joins/conditions/ordering. Bind variables (:F1, :H1, etc.) receive filter values. For large data, write optimized, indexed queries. Query zones usually handle pagination automatically.

Columns in Zones

Define what's displayed and how. Defined using type=sqlcol (e.g., Column1: type=sqlcol SQLCOL=PER_ID label=Person ID). Control column sequence and "Initial Columns to Display" (e.g., C1 C2 C5 C3 C4). Broadcast configured at column level (e.g., Broadcast: PER_ID=C1 ACCT_ID=C3) — passes selected row values to other zones. Also supports formatting, sorting, hyperlinks.

Pagination in Zones

Breaks large result sets into pages. Query Data Explorer zones usually support automatic pagination. For performance, use ROWNUM/ROW_NUMBER() in Oracle SQL and control rows per page. Info zones rarely need pagination (limited data). Always filter and limit data fetched for performance.

PAGENEXT, PAGEPREV, PAGINGKEY

Special variables for efficient custom pagination in Explorer Zones — fetch only what's needed for the current page rather than the whole result set.

- **PAGENEXT** — fetches the next page; passed as a hidden filter; used in WHERE to get records after the last record of the previous page.
- **PAGEPREV** — fetches the previous page; same idea in reverse, enabling bi-directional paging.
- **PAGINGKEY (most important)** — holds the unique key (or composite key) of the last record fetched; PAGENEXT/PAGEPREV use it to determine where the next/previous page starts. Instead of ROWNUM/OFFSET, we compare keys (e.g., WHERE PRIMARY_KEY > :PAGINGKEY) — much better performance on large tables.

■ **Interview Tip:** *PAGINGKEY + PAGENEXT/PAGEPREV is the recommended, performant way to implement pagination in OUAF Explorer Zones on large data volumes.*

Info Zone vs Query Zone

Both are Explorer Zones but serve different purposes — Info Zone displays information; Query Zone searches and lists records.

- **Use Info Zone when:** showing details of a record/context-driven info (Person details, Account Summary, Recent Bills), no need to search/filter a large list, read-only structured display, can sit on dashboards. Usually limited/fixed data, single or multiple SQL, light/no pagination.
- **Use Query Zone when:** user needs to search and see a list of records, needs pagination/sorting/multi-select/broadcasting (Person Search, Account List, Bill Search, To Do List). Heavy use of User/Hidden Filters, designed for large result sets, frequent broadcasting.

■ **Interview Tip:** *Show information → Info Zone. Search and list records → Query Zone.*

13. Portals

Main Purpose: A Portal is a display page made up of one or more Zones. Everything seen on screen (Person page, Account page, Dashboard) is basically a Portal.

Three Types:

- Standalone: full page linked to a menu. Used for main pages (Person, Account, Bill). Has its own Navigation Option and Menu entry.
- Dashboard: always visible on the right side of the screen. Used for quick info/summary views.
- Tab Page: groups multiple portals as tabs inside one page.

Key Points:

- Standalone is most common; automatically creates a Navigation Option when added; linked to menus; used for main transaction pages.
- Dashboard is always available regardless of which page the user is on.
- Tab Page organizes related content without creating multiple separate pages.
- Every Portal is built using Zones (Info or Query); we control the sequence/layout of zones inside it.

What gets created automatically when we create a Portal?

- Only for Standalone Portals: a Navigation Option **and** an Application Service are created automatically.
- Dashboard/Tab Page portals: nothing created automatically — Navigation Option/Application Service must be created manually if needed.
- Zones are never auto-created — created separately and added to the Portal.
- Menu entry is never auto-created — added manually to the required Menu.

■ **Interview Tip:** "Navigation Option and Application Service are created automatically for a Standalone Portal."

14. Navigation Option

Main Purpose: Used to navigate to a Portal, BPA Script, or any other page in the system — acts like a link/shortcut defining where the user should go.

Key Points:

- Auto-created when a Standalone Portal is created.
- Used to link the portal to a Menu, call it from BPA Scripts, or add it to Favorite Links.
- The bridge between Menus and Portals.
- For Dashboard/Tab Page portals, usually created manually.

15. Application Service

Main Purpose: Controls security and access to components in the system (Portals, BOs, Zones, Scripts, etc.) — defines what actions a user is allowed to perform.

Key Points:

- Auto-created along with the Navigation Option when a Standalone Portal is created.
- Acts as the security object controlling who can access that portal.
- Contains Access Modes: Read (view), Add (create), Change (modify), Delete (remove), Execute (run scripts/processes — mostly BPA Scripts/Business Services).
- Selective access is possible — some users only Read, others Add/Change.

How Security Works (Flow):

- Application Service holds Access Modes (Read/Add/Change/Delete/Execute) → defines allowed actions.
- User Group is linked to the Application Service → groups users with similar access.
- User belongs to a User Group → the actual person logging in.

Flow: Create Users → Create User Groups and add users → Link Application Services to User Groups with specific access modes → Only then does the user get access to the Portal/component.

Important Points:

- Without linking the Application Service to a User Group, the user can't access the portal even if it appears in the menu.
- One of the most important pieces for role-based security in OUAF/ORMB.
- Custom components (custom BOs, custom zones) can have Application Services created manually.

■ **Interview Tip:** Always mention the flow: Application Service → Access Modes → User Group → Users.

16. Menu (closely related)

Main Purpose: Organizes and displays navigation options so users can access Portals/pages — the main navigation structure.

Four Types:

- Admin Menu — configuration/setup screens (lookups, BOs, etc.). Used by implementers/admins.
- Main Menu — day-to-day master/transaction data (Person, Account, Bill, Payment). Most used by end users.
- Sub-Menu — menu inside another menu, for organizing large menus.
- Context Menu — dynamic menu based on the current record (e.g., right-click actions on a Person/Account).

Key Points:

- Menus are built using Navigation Options.
- A Standalone Portal's auto-created Navigation Option can then be added to a Menu.
- Admin Menu = Configuration; Main Menu = Transaction/Master Data; Context Menu = Record-specific actions.
- Menu access is controlled via Application Services linked to User Groups.

17. Business Service

Main Purpose: A Business Service (BS) retrieves or updates data from one or more tables in a reusable way. Acts as a service layer callable from Zones, UI Maps, Scripts, or other components — mainly used to fetch data via existing Query Zones or Java services without writing complex logic repeatedly.

Key Points:

- BS is not a Business Object — mainly for data retrieval, sometimes updates.
- Callable from Zones, UI Maps, BPA Scripts, Plug-In Scripts, etc.
- Two main types: Zone Based and Java Based.

1. Zone Based Business Service

- Uses an existing Query Zone (Data Explorer) to fetch data.
- We specify the Zone Name in the Business Service Schema.
- The BS internally calls the zone's SQL and returns the result.
- Useful for reusing a well-written query zone in scripts/UI Maps. Easy to create/maintain.

2. Java Based Business Service

- Uses Java code (Service Program) to retrieve/update data.
- More powerful and flexible than Zone-based BS.
- Used for complex logic, multiple table joins, or business rules.
- Can perform both read and update operations.
- Preferred when Zone-based BS isn't sufficient.

Service Program

Main Purpose: For a Java-based Business Service, the actual logic is written in a Service Program — a Java class containing the code to retrieve/update data. Business Service is the configuration; Service Program is the Java code behind it.

Key Points:

- Mandatory for Java-based Business Services.
- A Java class implementing the business logic.
- The Business Service configuration points to this Service Program.
- Receives input via the Business Service Schema and returns output in the same schema.
- Can do complex operations: multi-table fetches, applying business rules, calling other BOs/BS, performing updates.

Flow: Call Business Service (from Script/UI Map/Zone) → BS passes data to Service Program → Service Program executes Java logic → Result returned via the BS.

When to create one: Zone-based BS isn't enough; complex logic/multi-table access/business validations needed; want a reusable service callable from many places.

How to call a Zone from a Business Service

Answer: Define the Zone Name in the Business Service Schema.

- Applicable only to Zone-based Business Services.
- Go to Business Service → Schema tab → specify the Zone Name to call.

Example: `<zone mapField="ZONE_CD" default="CM_PersonSearchZone"/>`

- The Zone must already exist and be a Query Zone (Data Explorer).
- Once defined, the BS internally calls that zone's SQL and returns the data — reusing the existing query instead of writing new Java logic.

18. Scripts

Main Purpose: Scripts let us write business logic, automate processes, and perform operations without always writing Java code.

Four Types:

- Plug-In Script (Server Side) — attached to Algorithms; used inside Algorithm Types.
- Service Script (Server Side) — reusable logic for data operations; callable from many places.
- BPA Script (Client Side) — UI automation and navigation; screen navigation, button clicks.
- Groovy Script (Server Side) — complex logic and calculations; modern and powerful.

Key Points:

- Plug-In Script: plugged into Algorithm Types; runs at specific system events (validation, post-processing, etc.); receives parameters from the Algorithm.
- Service Script: standalone and reusable; callable from Plug-In Scripts, BPA Scripts, UI Maps, Zones; has its own Schema; best for reusable business logic.
- BPA Script: client side (browser); used for navigation, opening portals, clicking buttons, automating UI actions; can call other BPA Scripts, UI Maps, and invoke BO/BS/SS.
- Groovy Script: complex logic, commonly used in modern implementations (Oracle's cloud guidance recommends Groovy over Java for custom algorithm logic).

Important Differences:

- Server Side (Plug-In, Service, Groovy) rolls back on error; Client Side (BPA) may commit steps already done before an error occurs.
- Schema is present in Service Script and BPA Script (not in Plug-In Script).
- Server side → business logic & data operations; Client side → UI navigation & automation.

Plug-In Script vs Service Script

- **Main Purpose:** Plug-In runs inside Algorithms; Service Script is a standalone reusable script.
- **Attachment:** Plug-In must be attached to an Algorithm Type; Service Script is not attached to anything and can be called independently.
- **Schema:** Plug-In has no Schema (parameters come from the Algorithm); Service Script always has its own Schema.
- **Parameters:** Plug-In receives Hard Parameters (from the Algorithm Spot) and Soft Parameters; Service Script receives parameters through its own Schema.
- **Reusability:** Plug-In limited (mainly inside Algorithms); Service Script highly reusable (Plug-In Scripts, BPA Scripts, UI Maps, Zones).
- **Most Important Difference:** Plug-In Script has no schema; Service Script always has one.

Client Side vs Server Side Scripting

- **Server Side (Plug-In, Service, Groovy):** runs on the Application Server; business logic, data operations, validations; direct access to BO/BS/SQL indirectly; better for heavy processing; rolls back transaction on error; more secure (logic runs server-side).
- **Client Side (BPA):** runs on the user's browser; UI navigation, screen automation, button clicks; no direct DB access (must call server-side components); faster for UI actions; may commit prior steps on error; less secure (logic runs in the browser).
- **Most Important Difference:** Server-side scripts handle what happens with the data; client-side scripts handle how the user interacts with the screen.

Steps in a Script

Main Purpose: Steps are the building blocks inside a Script where the actual logic is written — each step performs one action (calling a BO/BS, navigating, etc.). Breaking logic into steps improves readability and control.

Key Points:

- Every script (BPA, Service, Plug-In) contains Steps.
- Each step has a Step Type; steps execute sequentially; conditional logic (If) can be added between steps.

Common Commands:

- `invokeBO` — call a Business Object (Add, Read, Update, Delete, FastAdd, etc.) — all script types.
- `invokeBS` — call a Business Service — all script types.
- `invokeSS` — call a Service Script — all script types.
- `invokeMap` — open/call a UI Map — BPA Script only.
- `performScript` — call another BPA Script (control returns after execution) — BPA Script only.
- `transferControl` — call another BPA Script (control does not return) — BPA Script only.
- `Navigate` — navigate to a Portal/BPA Script using a Navigation Option — BPA Script only.

Example (BPA Script):

```
Step 1: invokeBO 'Person' using "PersonBO" for read;
Step 2: if (PersonBO.PER_ID is not blank)
```

```
Step 3: invokeMap 'PersonUI' using "PersonMap";
Step 4: else
Step 5: Navigate 'PersonSearchPortal';
```

Important Points:

- Service/Plug-In Scripts mostly use invokeBO, invokeBS, invokeSS.
- BPA Scripts additionally use invokeMap, performScript, transferControl, Navigate.
- Steps keep scripts readable and maintainable; comments can be added for documentation.

Move Command (Edit Data Area)

Main Purpose: Copies data from one data area (or schema structure) to another inside a script — used in the Edit Data Area step. Instead of mapping fields one by one, we move whole data, groups, or lists in a single command.

Key Points:

- Used in the Edit Data Area step type.
- Transfers data between Data Areas, Schema structures, or temporary storage areas inside a script.
- Very useful with complex schemas containing groups/lists.

```
Syntax: move "SourceDataArea" to "TargetDataArea";
```

Examples:

```
Move entire data area: move "PersonInput" to "PersonBO";
Move specific group: move "PersonInput.name" to "PersonBO.personName";
Move list data: move "AccountList" to "AccountBO.accountList";
```

When to use: To avoid multiple individual field mappings, when working with nested groups/lists, to keep scripts clean — commonly used in Service/BPA Scripts when preparing data before calling a BO/BS.

■ **Interview Tip:** Move command is used in the Edit Data Area step; different from mapField (used in BO Schema).

Groovy Scripting

Main Purpose: Used to write more powerful, flexible, and complex business logic compared to traditional Plug-In/Service Scripts — runs on the JVM.

Key Points:

- Groovy is a dynamic language fully compatible with Java.
- Used in Algorithm Types (as Groovy Algorithm), Service Scripts, and some advanced BO customizations.
- Especially useful when logic is too complex for normal script steps or XPATH.
- Oracle's cloud implementation guidance specifically recommends writing new algorithm logic in Groovy (or XML-based scripting) rather than custom Java, since custom Java algorithm types aren't permitted in cloud services.

When to use Groovy: complex calculations/business rules; heavy use of loops, conditions, collections (List, Map); working with multiple records/complex data structures; better readability/maintainability; reusing Java-like constructs for advanced logic.

Advantages over traditional scripts: full programming syntax (vs step-based/limited); much easier for complex logic; strong collection/loop support; better readability; better performance for complex processing; slightly higher learning curve but very powerful.

Where commonly used: complex validation/post-processing logic in Algorithms; advanced business rules in Service Scripts; bulk data processing/transformations; cases where normal script steps become too long to manage.

■ **Interview Tip:** *"Groovy is used when we need complex business logic that's hard to implement using normal Plug-In or Service Scripts — it provides loops, collections, and better structure, while still running server-side."*

19. UI Maps

Main Purpose: A UI Map is used to create custom HTML screens in OUAF/ORMB — designing UIs using HTML, CSS, and JavaScript instead of standard framework screens. Useful when standard zones/portals aren't sufficient and a more customized/user-friendly screen is needed.

Four Types:

- Complete HTML Document — full screen UI Map; for main screens/complete pages.
- Complete XHTML Document — full screen UI Map (XHTML format); for main screens/complete pages.
- HTML Fragment — small reusable section; embedded inside a complete UI Map.
- XHTML Fragment — small reusable section (XHTML format); embedded inside a complete UI Map.

Key Points:

- Complete HTML/XHTML Document → full screens.
- HTML/XHTML Fragment → small reusable sections included inside a complete UI Map.

Important Features:

- Every UI Map has a Schema defining the data structure (input/output fields).
- We generate the HTML screen using the schema.
- Custom HTML, CSS, and JavaScript for better UI/UX.
- Can call Business Objects, Business Services, and Service Scripts from the UI Map.

Important Commands:

- `orainvokeBO` — call a Business Object. Example: `orainvokeBO('Person', 'read');`
- `orainvokeBS` — call a Business Service. Example: `orainvokeBS('CM_GetAccount');`
- `orainvokeSS` — call a Service Script. Example: `orainvokeSS('CM_ValidateData');`
- `orainvokeScript` — call a BPA Script. Example: `orainvokeScript('CM_NavigateToPage');`

When to Use UI Maps: need a highly customized screen; standard zones/portals aren't flexible enough; want reusable UI components (Fragments); need heavy JavaScript for dynamic behavior; want full control over the interface.

■ **Interview Tip:** Use Complete HTML Document for full pages, HTML Fragment for reusable small sections. Always mention that UI Maps contain a Schema, and that BO/BS/SS are called using `orainvokeBO`, `orainvokeBS`, `orainvokeSS`, `orainvokeScript`.